MATERIAL DIDÁCTICO · ACCESO A DATOS

TUTORIAL RESUELTO 03 · UNIDAD 02

Hibernate ORM paso a paso

Maven, entidades, asociaciones, sesiones, transacciones, consultas JPQL y verificación.

Objetivo y preparación

Configurar Hibernate para representar las tablas SQL como clases Java persistentes. Reutilizamos MySQL y el esquema ad_ud2 creado en T01, pero ahora las operaciones se expresan mediante Session, entidades y consultas JPQL.

Paso 1. Arrancar el servidor y abrir la carpeta Maven

TERMINAL

```
cd Codigo/Infraestructura/mysql
docker compose up -d

# Después abre otra terminal:
cd Codigo/T03_Hibernate_ORM
mvn clean compile exec:java
```

En el IDE importa la carpeta T03_Hibernate_ORM. No hace falta descargar manualmente Hibernate: el pom.xml solicita las dependencias a Maven. La primera compilación requiere Internet.

Paso 2. Configurar las dependencias

2.1. pom.xml con Hibernate y MySQL

Ruta: **Codigo/T03_Hibernate_ORM/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>t03-hibernate-orm</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
    </dependency>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.6.3.Final</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
```

```
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.t03.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

La versión Hibernate 6.6.3.Final emplea anotaciones jakarta.persistence, no javax.persistence. Con Java 17 el proyecto es compatible.

Paso 3. Mapear las entidades

Creamos una clase Club que corresponde a la tabla club y otra Jugador vinculada con una relación ManyToOne. El constructor sin argumentos es necesario para que Hibernate pueda construir entidades; los getters y setters ofrecen acceso a sus propiedades.

3.1. Entidad Club

Ruta: **Codigo/T03_Hibernate_ORM/src/main/java/es/curso/ad/t03/Club.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t03;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.math.BigDecimal;

@Entity
@Table(name = "club")
public class Club {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true, length = 90)
    private String nombre;

    @Column(nullable = false, length = 90)
    private String ciudad;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal presupuesto;

    protected Club() { }

    public Club(String nombre, String ciudad, BigDecimal presupuesto) {
        this.nombre = nombre;
        this.ciudad = ciudad;
        this.presupuesto = presupuesto;
    }
```

```
    public Integer getId() { return id; }
    public String getNombre() { return nombre; }
    public String getCiudad() { return ciudad; }
    public BigDecimal getPresupuesto() { return presupuesto; }
    public void setCiudad(String ciudad) { this.ciudad = ciudad; }

    @Override
    public String toString() {
        return id + " | " + nombre + " | " + ciudad + " | " + presupuesto;
    }
}
```

3.2. Entidad Jugador

Ruta: **Codigo/T03_Hibernate_ORM/src/main/java/es/curso/ad/t03/Jugador.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t03;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "jugador")
public class Jugador {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String nombre;
    private int dorsal;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "club_id", nullable = false)
    private Club club;

    protected Jugador() { }

    public Jugador(String nombre, int dorsal, Club club) {
        this.nombre = nombre;
        this.dorsal = dorsal;
        this.club = club;
    }

    public Integer getId() { return id; }
    public String getNombre() { return nombre; }
    public int getDorsal() { return dorsal; }
    public Club getClub() { return club; }
}
```

El atributo club representa una asociación Java y @JoinColumn define dónde se almacena la referencia SQL. No creamos una columna clubId independiente en la misma entidad.

Paso 4. Crear la SessionFactory

Centralizamos la configuración de Hibernate. La SessionFactory es costosa y debe compartirse durante la ejecución. La opción validate impide que un ejemplo didáctico cree o borre tablas sin avisar.

4.1. HibernateUtil.java

Ruta: **Codigo/T03_Hibernate_ORM/src/main/java/es/curso/ad/t03/HibernateUtil.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t03;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public final class HibernateUtil {
    private HibernateUtil() { }

    public static SessionFactory crear() {
        Configuration cfg = new Configuration();
        cfg.setProperty("hibernate.connection.driver_class", "com.mysql.cj.jdbc.Driver");
        cfg.setProperty("hibernate.connection.url",
                "jdbc:mysql://localhost:3307/ad_ud2?connectionTimeZone=UTC");
        cfg.setProperty("hibernate.connection.username", "ad_user");
        cfg.setProperty("hibernate.connection.password", "ad_pass");
        cfg.setProperty("hibernate.hbm2ddl.auto", "validate");
        cfg.setProperty("hibernate.show_sql", "true");
        cfg.addAnnotatedClass(Club.class);
        cfg.addAnnotatedClass(Jugador.class);
        return cfg.buildSessionFactory();
    }
}
```

Paso 5. Programar el DAO ORM

En listar usamos HQL; en jugadoresDeCiudad hacemos join entre entidades; en cambiarCiudad cargamos una entidad managed, modificamos un campo y confirmamos la transacción; en anadirJugador recuperamos el club y persistimos un nuevo jugador.

5.1. ClubOrmDao.java

Ruta: **Codigo/T03_Hibernate_ORM/src/main/java/es/curso/ad/t03/ClubOrmDao.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t03;

import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

public class ClubOrmDao {
    private final SessionFactory factoria;
```

```
    public ClubOrmDao(SessionFactory factoria) {
        this.factoria = factoria;
    }

    public List<Club> listar() {
        try (Session sesion = factoria.openSession()) {
            return sesion.createQuery("from Club c order by c.id", Club.class)
                    .getResultList();
        }
    }

    public List<String> jugadoresDeCiudad(String ciudad) {
        try (Session sesion = factoria.openSession()) {
            return sesion.createQuery(
                    "select j.nombre from Jugador j join j.club c " +
                    "where c.ciudad = :ciudad order by j.nombre", String.class)
                    .setParameter("ciudad", ciudad)
                    .getResultList();
        }
    }

    public boolean cambiarCiudad(int id, String nuevaCiudad) {
        try (Session sesion = factoria.openSession()) {
            Transaction transaccion = sesion.beginTransaction();
            try {
                Club club = sesion.find(Club.class, id);
                if (club == null) {
                    transaccion.rollback();
                    return false;
                }
                club.setCiudad(nuevaCiudad); // Dirty checking: no UPDATE manual
                transaccion.commit();
                return true;
            } catch (RuntimeException e) {
                if (transaccion.isActive()) transaccion.rollback();
                throw e;
            }
        }
    }

    public int anadirJugador(int clubId, String nombre, int dorsal) {
        try (Session sesion = factoria.openSession()) {
            Transaction transaccion = sesion.beginTransaction();
            try {
                Club club = sesion.find(Club.class, clubId);
                if (club == null) {
                    throw new IllegalArgumentException("Club no encontrado");
                }
                Jugador jugador = new Jugador(nombre, dorsal, club);
                sesion.persist(jugador);
                transaccion.commit();
                return jugador.getId();
            } catch (RuntimeException e) {
                if (transaccion.isActive()) transaccion.rollback();
                throw e;
            }
        }
    }
}
```

Si cambias la ciudad mientras el objeto está asociado a la sesión, Hibernate detecta la modificación. No escribimos UPDATE manual en ese método.

5.2. Main.java

Ruta: **Codigo/T03_Hibernate_ORM/src/main/java/es/curso/ad/t03/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t03;

import org.hibernate.SessionFactory;

public class Main {
    public static void main(String[] args) {
        try (SessionFactory factoria = HibernateUtil.crear()) {
            ClubOrmDao dao = new ClubOrmDao(factoria);
            System.out.println("CLUBES CON HIBERNATE");
            dao.listar().forEach(System.out::println);
            System.out.println("Jugadores de Granada: " +
                    dao.jugadoresDeCiudad("Granada"));
            System.out.println("Actualizar ID 1: " +
                    dao.cambiarCiudad(1, "Granada"));

            // Una sola vez, si se desea probar INSERT:
            // System.out.println("Jugador ID: " +
            //         dao.anadirJugador(1, "Iván", 11));
        } catch (RuntimeException e) {
            System.err.println("Error ORM: " + e.getMessage());
        }
    }
}
```

El Main imprime la lista y los jugadores de Granada. La inserción opcional está comentada: si la activas, utiliza un dorsal libre y comprueba que la restricción UNIQUE se mantiene.

Paso 6. Comprobar el SQL generado

Ejecuta **mvn clean compile exec:java**. Debes ver instrucciones SQL que Hibernate produce y, después, la salida del Main. Si aparece un error relacionado con el esquema, asegúrate de que las tablas del tutorial T01 existen. No actives create ni create-drop para intentar corregirlo sin revisar la causa.

Paso 7. Practicar los estados de una entidad

- Localiza el objeto nuevo Jugador antes de persist: es transient.
- Después de persist en una sesión abierta, la entidad pasa a estar managed.
- Cierra la sesión y razona por qué el objeto ya no está managed.
- Identifica el rollback en caso de excepción y explica por qué no basta con cerrar la sesión.

Paso 8. Comprobar la relación

Desde MySQL ejecuta **SELECT * FROM jugador;**. Después vuelve a la consulta JPQL con join. Comprueba que devuelve nombres de entidades asociadas a Club, sin que aparezca en el código JPQL la columna física club_id.

Advertencia sobre ORM
Hibernate simplifica la persistencia de objetos, pero no sustituye el conocimiento de SQL. Antes de dar por válida una consulta, revisa su resultado y las sentencias generadas.